DISEÑO DE COMPILADORES



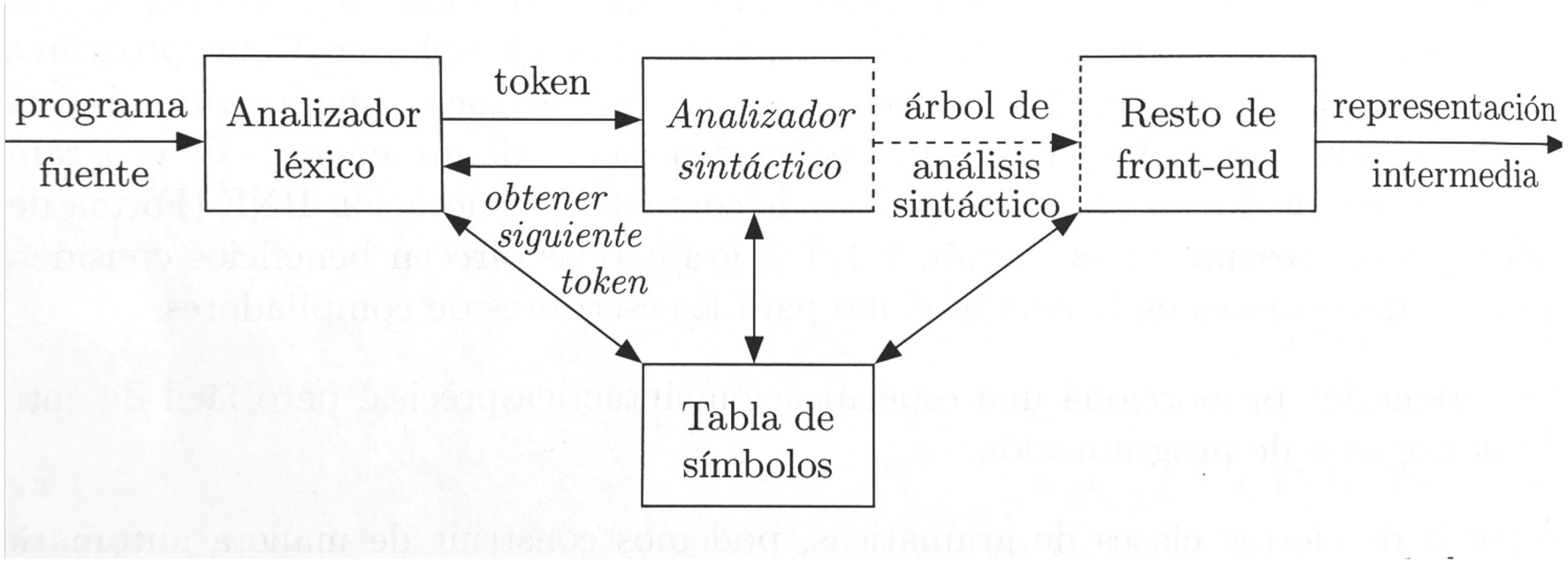
Capítulo 4

Análisis Sintáctico

Observación. *El presente documento sirve de complemento al contenido que referencia y no se constituye en el detalle del mismo, ya que conceptos y contenidos pueden haber sido desarrollados en forma presencial, virtual o asignado como ejercicio. Ante cualquier duda se recomienda consultar con el profesor a cargo de la cátedra.*

Profesor: Sergio A. Aranda

ANALIZADOR SINTÁCTICO
Introducción



DEFINICIÓN FORMAL DE UNA GRAMATICA LIBRE DE CONTEXTO

4.2.1 La definición formal de una gramática libre de contexto

En la sección 2.2 vimos que una gramática libre de contexto (o simplemente gramática) consiste en terminales, no terminales, un símbolo inicial y producciones.

1. Los *terminales* son los símbolos básicos a partir de los cuales se forman las cadenas. El término "nombre de token" es un sinónimo de "terminal"; con frecuencia usaremos la palabra "token" en vez de terminal, cuando esté claro que estamos hablando sólo sobre el nombre del token. Asumimos que las terminales son los primeros componentes de los tokens que produce el analizador léxico. En (4.4), los terminales son las palabras reservadas **if** y **else**, y los símbolos "(" y ")".

2. Los *no terminales* son variables sintácticas que denotan conjuntos de cadenas. En (4.4), *instr* y *expr* son no terminales. Los conjuntos de cadenas denotados por los no terminales ayudan a definir el lenguaje generado por la gramática. Los no terminales imponen una estructura jerárquica sobre el lenguaje, que representa la clave para el análisis sintáctico y la traducción.

3. En una gramática, un no terminal se distingue como el *símbolo inicial*, y el conjunto de cadenas que denota es el lenguaje generado por la gramática. Por convención, las producciones para el símbolo inicial se listan primero.

4. Las producciones de una gramática especifican la forma en que pueden combinarse los terminales y los no terminales para formar cadenas. Cada *producción* consiste en:

   (a) Un no terminal, conocido como *encabezado* o *lado izquierdo* de la producción; esta producción define algunas de las cadenas denotadas por el encabezado.

   (b) El símbolo $\rightarrow$. Algunas veces se ha utilizado `::=` en vez de la flecha.

   (c) Un *cuerpo* o *lado derecho*, que consiste en cero o más terminales y no terminales. Los componentes del cuerpo describen una forma en que pueden construirse las cadenas del no terminal en el encabezado.

Instr $\rightarrow$ **if** (expr) instr **else** instr

DERIVACIONES

$$E \rightarrow E + E \mid E * E \mid - E \mid (E) \mid \mathbf{id}$$

Derivación por la izquierda (leftmost)

$$E \underset{lm}{\Rightarrow} -E \underset{lm}{\Rightarrow} -(E) \underset{lm}{\Rightarrow} -(E + E) \underset{lm}{\Rightarrow} -(\mathbf{id} + E) \underset{lm}{\Rightarrow} -(\mathbf{id} + \mathbf{id})$$

Derivación por la derecha (rightmost)

$$E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(E + E) \Rightarrow -(E + \mathbf{id}) \Rightarrow -(\mathbf{id} + \mathbf{id})$$

Una derivación puede derivar en cero o más pasos a una cadena de terminales y no terminales. Entonces:

1. *α =>* α,*
2. *Si α =>* ß, y ß=>*∂, entonces α =>* ∂.*

Observación. Es la secuencia de recorrida de un árbol sintáctico.

DERIVACIONES
Ejercicios

[Aho] Considere las siguientes gramáticas libre de contexto y :

a. Proporcione una derivación por la izquierda para la cadena,
b. Proporcione una derivación por la derecha para la cadena,
c. Un árbol de análisis sintáctico para la cadena,
d. ¿La gramática es ambigua? Justifique su respuesta.

S -> (L) | a
L -> L, S | S
con la cadena ((a, a), a, (a))

bexpr -> bexpr or bterm | bterm
bterm -> bterm and bfactor | bfactor
bfactor -> not bfactor | (bexpr) | true | false
(proponer la cadena)

[Aho] Diseñar una gramática que represente el conjunto de todas las cadenas de 0 y 1 que sean palíndromas, es decir, que la cadena se lea igual al derecho y al revés.

Ejemplo: 01010

RELACION BNF – EXPRESIONES REGULARES

Conversión

Gramáticas libres de contexto: Notación más poderosa que las expresiones regulares.

Algoritmo de conversión: A partir de una expresión regular se puede encontrar una gramática BNF equivalente.

Entrada: Una expresión regular.
Salida: Un BNF equivalente.

METODO

1. A partir de la expresión regular de entrada, obtener el AFN equivalente. Opcionalmente se obtiene el AFD y/o el AFD mínimo equivalente.
2. Para cada estado *i* del AFN, crear un no terminal A_i.
3. Si el estado *i* tienen una transición al estado *j* con la entrada **a**, agregar la producción $A_i \rightarrow aA_j$. Si el estado i pasa al estado j con la entrada ε, agregar la producción $A_i \rightarrow A_j$.
4. Si *i* es un estado de aceptación o final, agregar $A_i \rightarrow \varepsilon$.
5. Si *i* es el estado inicial, hacer que A_i sea el símbolo inicial del BNF.

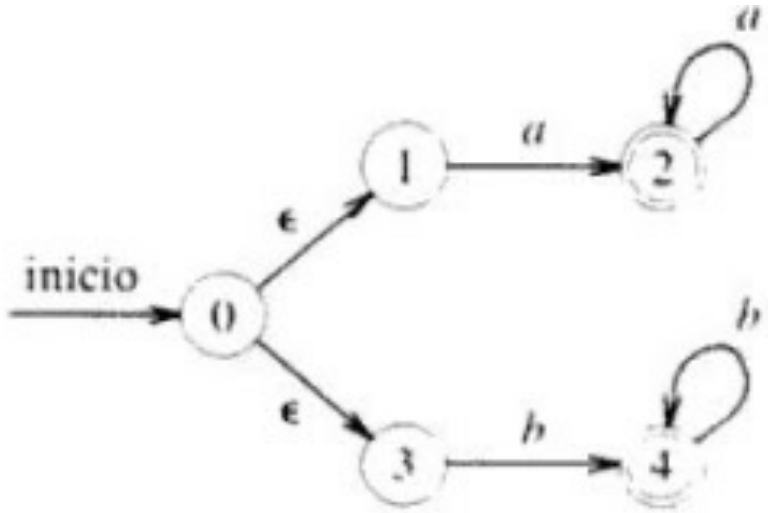


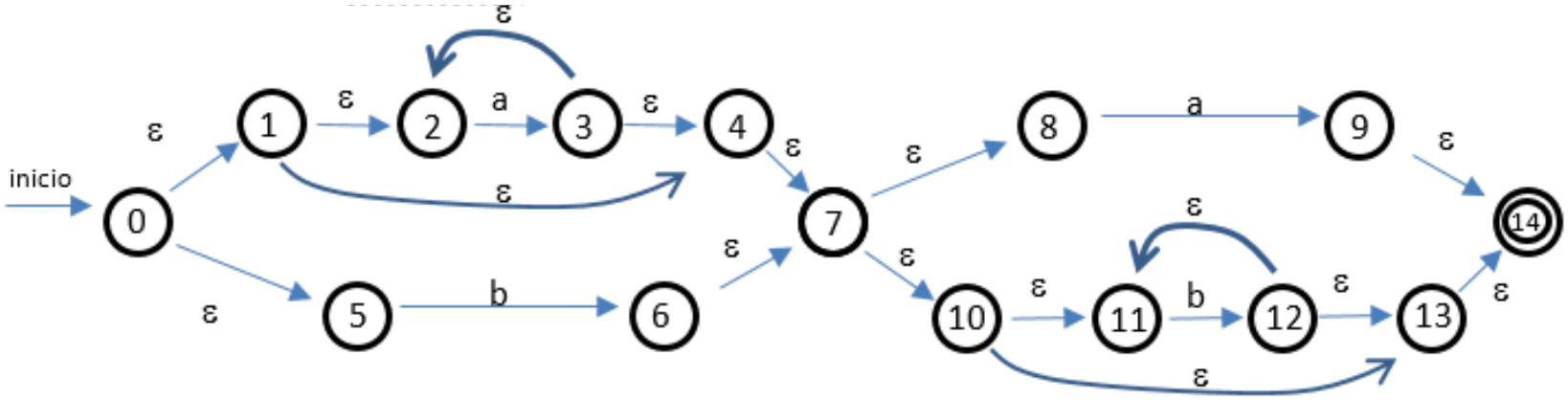
Fig. 3.21. Un AFN que acepta $aa^* \mid bb^*$.

Gramática original:

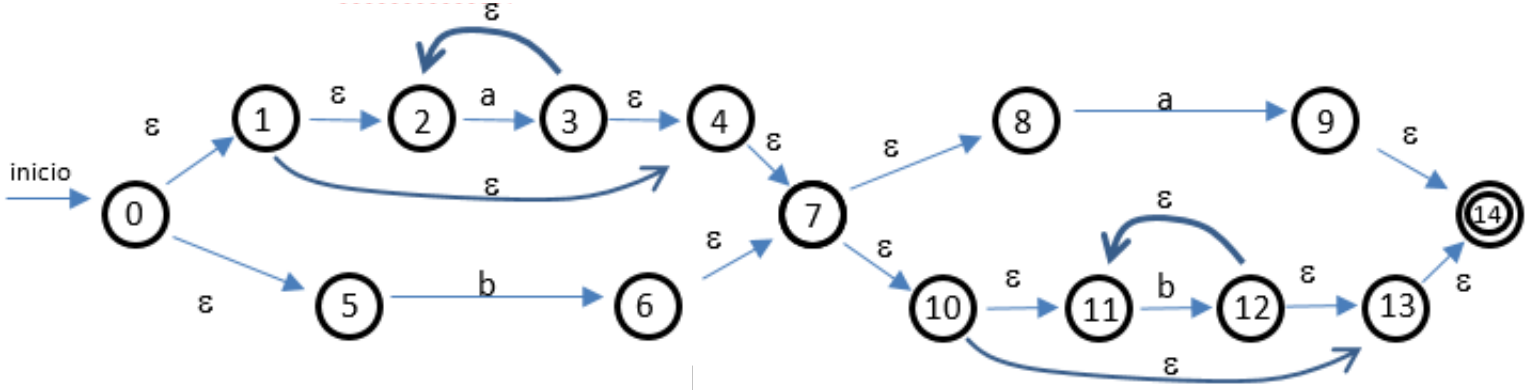
A0 -> A1 | A3
A1 -> a A2
A3 -> bA4
A2 -> aA2 | ε
A4 -> bA4 | ε

Gramática simplificada:

A0 -> aA2 | bA4
A2 -> aA2 | ε
A4 -> bA4 | ε



(a* | b) (a | b*)



$A_0 \rightarrow A_1 \mid A_5$
$A_1 \rightarrow A_2 \mid A_9$
$A_2 \rightarrow a A_3$
$A_3 \rightarrow A_2 \mid A_4$
$A_4 \rightarrow A_7$
$A_5 \rightarrow b A_6$
$A_6 \rightarrow A_7$
$A_7 \rightarrow A_8 \mid A_{10}$
$A_8 \rightarrow a A_9$
$A_9 \rightarrow A_{14}$
$A_{10} \rightarrow A_{11} \mid A_{13}$

$A_{11} \rightarrow b A_{12}$
$A_{12} \rightarrow A_{13} \mid A_{11}$
$A_{13} \rightarrow A_{14}$
$A_{14} \rightarrow \epsilon$

...
A1-> A2 | A4
A1 -> a A3 | aA9 | b A12| ε
...

Pasos: 1. expr reg (creatividad)
2. Thompson para AFN equivalente
3. Subconjunto para AFD eq.
4. Minimización para AFD min eq.
5. Algoritmo conversion AFD -> BNF

(a|b)*abb, probar con AFN Thompson, y con el AFD min obtenido.

ELIMINACIÓN DE LA REC. POR LA IZQUIERDA
Recursión indirecta

Algoritmo 4.19: Eliminación de la recursividad por la izquierda.

ENTRADA: La gramática G sin ciclos ni producciones ϵ.

SALIDA: Una gramática equivalente sin recursividad por la izquierda.

MÉTODO: Aplicar el algoritmo de la figura 4.11 a G. Observe que la gramática no recursiva por la izquierda resultante puede tener producciones ϵ. □

1) ordenar los no terminales de cierta forma $A_1, A_2, \ldots, A_n$.
2) **for** (cada i de 1 a n) {
3) **for** (cada j de 1 a $i-1$) {
4) sustituir cada producción de la forma $A_i \rightarrow A_j\gamma$ por las producciones $A_i \rightarrow \delta_1\gamma \mid \delta_2\gamma \mid \cdots \mid \delta_k\gamma$, en donde $A_j \rightarrow \delta_1 \mid \delta_2 \mid \cdots \mid \delta_k$ sean todas producciones A_j actuales
5) }
6) eliminar la recursividad inmediata por la izquierda entre las producciones A_i
7) }

Figura 4.11: Algoritmo para eliminar la recursividad por la izquierda de una gramática

ELIMINACIÓN DE LA RECURSIVIDAD POR LA IZQUIERDA

$$S \rightarrow Aa \mid b$$
$$A \rightarrow Ac \mid Sd \mid \epsilon$$

ELIMINACIÓN DE LA RECURSIVIDAD POR LA IZQUIERDA

$$S \rightarrow Aa \mid b$$
$$A \rightarrow Ac \mid Sd \mid \epsilon$$

$$\downarrow$$

(1) $S \rightarrow Aa \mid b$

---

(2) $A \rightarrow Ac \mid Aad \mid bd \mid \epsilon$

$A \rightarrow A\alpha \mid \beta \rightarrow \begin{cases} A \rightarrow \beta R \\ R \rightarrow \alpha R \mid \epsilon \end{cases}$ $\therefore$ $\begin{array}{l} A \rightarrow bdR \mid R \\ R \rightarrow cR \mid adR \mid \epsilon \end{array}$

ELIMINACIÓN DE LA RECURSIVIDAD POR LA IZQUIERDA

Ordenar el sistema de producciones:

A-> Ac|Sd|ε

S-> Aa | b

Para i=1, j=1-i=0,

A-> Ac|Sd|ε, aplicar la rec. Izq directa, y queda:

A-> SdR | R

R-> cR | ε

Para i=2, j=1

S-> Aa | b, sustituir con A_1=A, y queda:

S-> SdRa | Ra | b,

Eliminar Rec izq directa:

S-> RaR_2 | bR_2

R_2->$dRaR_2$ | ε

Finalmente queda:

A-> SdR | R

R-> cR | ε

S-> RaR_2 | bR_2

R_2->$dRaR_2$ | ε

PRIMERO (X)

1. Si X es un terminal, entonces PRIMERO(X) = {X}
2. Si X es un no terminal y $X \rightarrow Y_2Y_1 \ldots Y_k$ es una producción para $k \geq 1$, entonces se agrega **a** en PRIMERO(X) si para cierta i, **a** está en PRIMERO(Y_i), y ε está en todos las funciones PRIMERO(Y_1) … PRIMERO(Y_{i-1})
3. Si ε esta en todos los conjuntos PRIMERO(Y_1) a PRIMERO(Y_k), entonces se agrega ε en PRIMERO(X).
4. Si $X \rightarrow \varepsilon$ es una producción definida, entonces se agrega ε a PRIMERO(X).

PRIMERO ($X_1X_2 \ldots X_n$)

1. Agregar a PRIMERO($X_1X_2 \ldots X_n$) todos los símbolos que no sean ε de PRIMERO(X_1).
2. Si ε está en PRIMERO(X_1), agregar todos los símbolos que no sean ε de PRIMERO(X_2).
3. Si ε está en PRIMERO(X_1) y PRIMERO(X_2), entonces agregar todos los símbolos que no sean ε de ambos, y así sucesivamente.
4. Si ε está en todos los conjuntos PRIMERO de X_1 a X_n, entonces agregar ε al conjunto PRIMERO($X_1X_2 \ldots X_n$)

SIGUIENTE(X)
Aplicar las siguientes reglas hasta que no sea posible agregar ningún símbolo terminal nuevo a cualquier conjunto SIGUIENTE:

1. Colocar $ en SIGUIENTE(S), en donde S es el símbolo inicial y $ es el delimitador derecho de entrada (fin de cadena).

2. Para cada producción de la forma $A \rightarrow \alpha B \beta$, agregar a SIGUIENTE(B) todos los símbolos terminales a excepción de ε que se encuentren en PRIMERO(β).

3. Para toda producción de la forma:
   - $A \rightarrow \alpha B$, o,
   - $A \rightarrow \alpha B \beta$, en donde PRIMERO($\beta$) contiene a ε,

entonces todo lo que hay en SIGUIENTE(A) está en SIGUIENTE(B).

ANALISIS SINTÁCTICO DESCENDENTE

SIGUIENTE (X)

$$
\begin{aligned}
E &\rightarrow T\ E' \\
E' &\rightarrow +\ T\ E' \mid \epsilon \\
T &\rightarrow F\ T' \\
T' &\rightarrow *\ F\ T' \mid \epsilon \\
F &\rightarrow (\ E\ ) \mid \mathbf{id}
\end{aligned}
$$

SIGUIENTE

S(E)={ $,) }

S(E´)={ $,) }

S(T)={+, $,) }

S(T´)= {+, $,) }

S(F)={*, +, $,) }

PRIMERO

P(E)=P(TE´)={ (, id)

P(E´)={+} U {ε} = {+, ε }

Ejercicio 2:

A->aB | ε

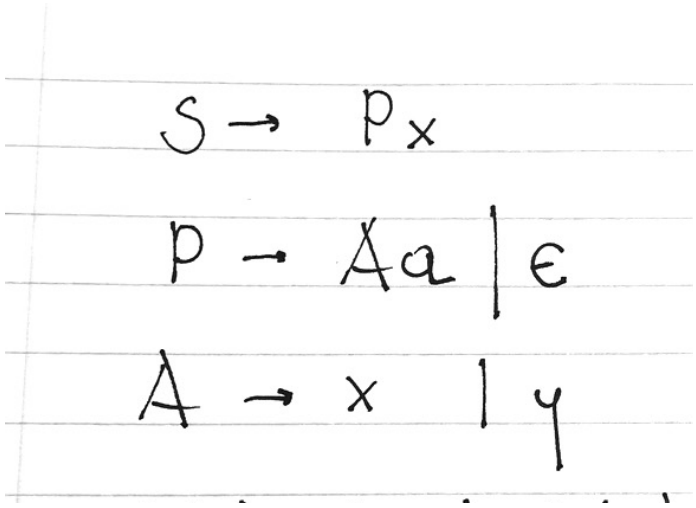
B->a | b

P(A)= {a} U {ε} = {a, ε}

P(B)= {a} U {b} = {a.b}

S(A)= {$}

S(B)= {$}


S → Px
P → Aa | ε
A → x | y

ANALISIS SINTÁCTICO PREDICTIVO

Gramáticas LL(1)

L L (1) →

Utilizar un símbolo de la cadena de entrada para realizar la selección de la acción sintáctica que corresponda (elegir la siguiente producción).

A partir del símbolo inicial, se produce una derivación por la izquierda para la verificación de pertenencia de la cadena al la gramática.

Se refiere al análisis de la cadena de entrada de izquierda a derecha, hasta encontrar el símbolo $.

ANALISIS SINTÁCTICO PREDICTIVO

Gramáticas LL(1)

Una gramática G con su Tabla de Análisis Sintáctico sin entradas múltiples se define como LL(1) y cumple con las siguientes condiciones:

a. No es recursiva por la izquierda
b. No hay ambiguedad:
   - Sintáctica, es decir está factorizada por la izquierda en caso de ser necesario
   - Semántica, es decir no hay ambigüedad en PRIMERO ni SIGUIENTE

O bien en otras palabras: Para toda $A \rightarrow \alpha \mid \beta$, se debe cumplir :

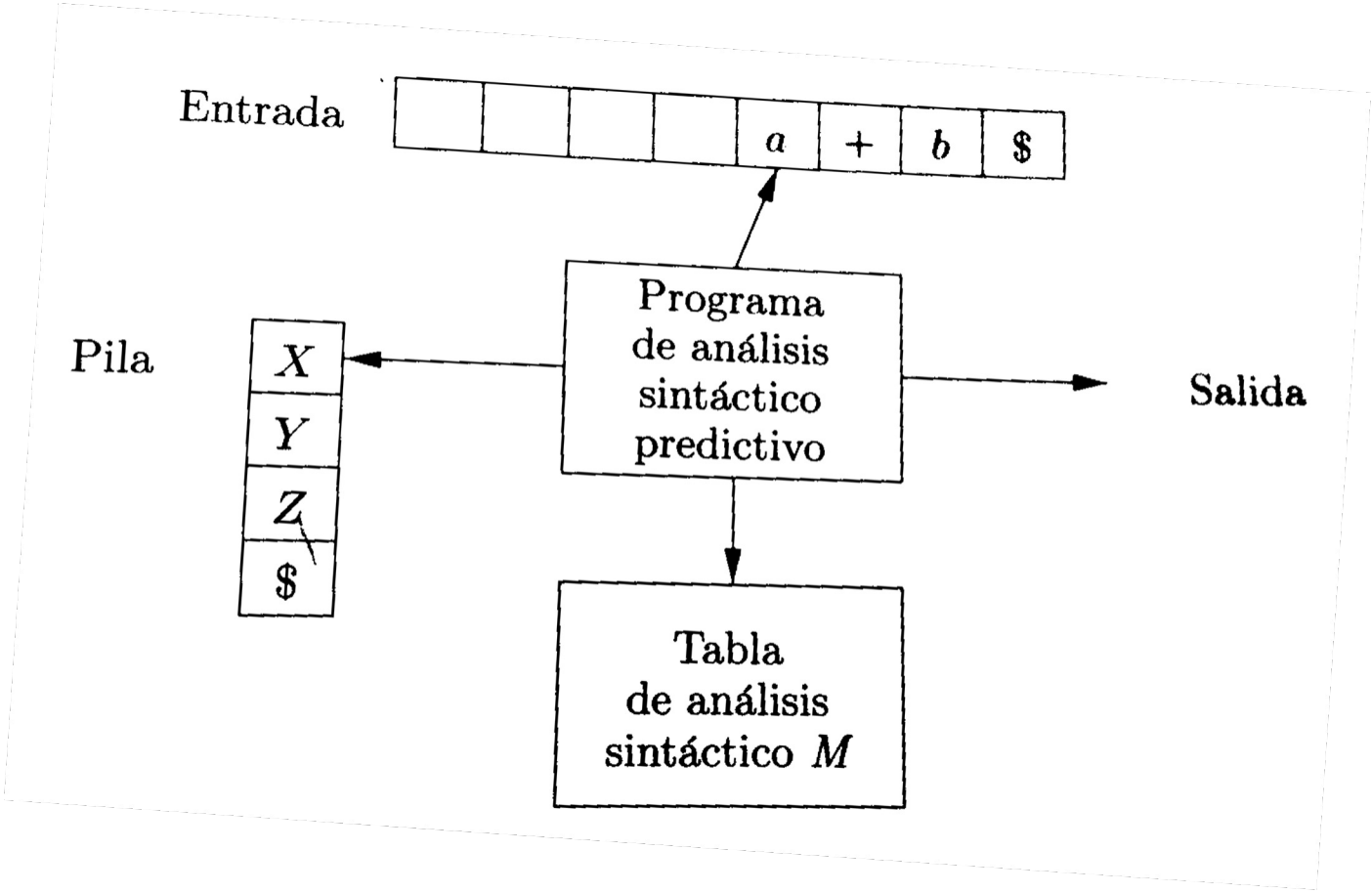
- Sólo α ó β a la vez puede derivar en ε.
- PRIMERO(α) es disjunto al PRIMERO(β)
- Si β deriva en ε (ß=>* ε), entonces α no deriva en ninguna cadena que comience con un terminal en SIGUIENTE(A).

Si una gramática no cumple con alguna de estas condiciones, hay que tratar de convertirla ya sea aplicando algoritmos de eliminación de recursión por la izquierda, factorizando o eliminando la ambigüedad.

Observación: No todas las gramáticas pueden ser convertidas en gramáticas LL(1).

ANALISIS SINTÁCTICO PREDICTIVO

Gramáticas LL(1)



Algoritmo 4.31: Construcción de una tabla de análisis sintáctico predictivo.

ENTRADA: La gramática G.

SALIDA: La tabla de análisis sintáctico M.

MÉTODO: Para cada producción $A \rightarrow \alpha$ de la gramática, hacer lo siguiente:

1. Para cada terminal a en PRIMERO(A), agregar $A \rightarrow \alpha$ a $M[A, a]$.
2. Si ϵ está en PRIMERO(α), entonces para cada terminal b en SIGUIENTE(A), se agrega $A \rightarrow \alpha$ a $M[A, b]$. Si ϵ está en PRIMERO(α) y \$ se encuentra en SIGUIENTE(A), se agrega $A \rightarrow \alpha$ a $M[A, \$]$ también.

$$
\begin{array}{lcl}
E & \rightarrow & T\ E' \\
E' & \rightarrow & +\ T\ E' \mid \epsilon \\
T & \rightarrow & F\ T' \\
T' & \rightarrow & *\ F\ T' \mid \epsilon \\
F & \rightarrow & (\ E\ ) \mid \mathbf{id}
\end{array}
$$

ANALISIS SINTÁCTICO PREDICTIVO

Gramáticas LL(1)

MÉTODO: Para cada producción $A \rightarrow \alpha$ de la gramática, hacer lo siguiente:

1. Para cada terminal a en PRIMERO(A), agregar $A \rightarrow \alpha$ a $M[A, a]$.
2. Si ϵ está en PRIMERO(α), entonces para cada terminal b en SIGUIENTE(A), se agrega $A \rightarrow \alpha$ a $M[A, b]$. Si ϵ está en PRIMERO(α) y \$ se encuentra en SIGUIENTE(A), se agrega $A \rightarrow \alpha$ a $M[A, \$]$ también.

$$
\begin{aligned}
E &\rightarrow T\,E' \\
E' &\rightarrow +\,T\,E' \mid \epsilon \\
T &\rightarrow F\,T' \\
T' &\rightarrow *\,F\,T' \mid \epsilon \\
F &\rightarrow (\,E\,) \mid \text{id}
\end{aligned}
$$

	id	(	)	*	+	$
E	E->TE´	E->TE´				
E´			E´-> ε		E´->+TE´	E´->ε
T	T->FT´	T->FT´				
T´			T´-> ε	T´->*FT´	T´-> ε	T´-> ε
F	F-> id	F-> (E)				

SIGUIENTE

S(E)={ \$,) }
S(E´)={ \$,) }
S(T)={+, \$,) }
S(T´)= {+, \$,) }
S(F)={*, +, \$,) }

ANALISIS SINTÁCTICO PREDICTIVO

Gramáticas LL(1)

No Terminal	Símbolo de Entrada					
	id	+	*	(	)	$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow$ id			$F \rightarrow (E)$		

ANALISIS SINTÁCTICO PREDICTIVO

Gramáticas LL(1)

MÉTODO: Para cada producción $A \rightarrow \alpha$ de la gramática, hacer lo siguiente:

1. Para cada terminal a en PRIMERO(A), agregar $A \rightarrow \alpha$ a $M[A, a]$.
2. Si ϵ está en PRIMERO(α), entonces para cada terminal b en SIGUIENTE(A), se agrega $A \rightarrow \alpha$ a $M[A, b]$. Si ϵ está en PRIMERO(α) y \$ se encuentra en SIGUIENTE(A), se agrega $A \rightarrow \alpha$ a $M[A, \$]$ también.

S -> i E t S S´| a
S´-> e S´ | ε
E -> b

	i	t	a	e	b	$
S	S -> i E t S S´		S-> a			
S´				S´->eS´ S´-> ε		S´-> ε
E					E->b	

P(S)= {i} U {a}
P(S´)={e} U {ε}
P(E)={b}

S(S)={$,e}
S(S´)={$,e}
S(E)={t}

MÉTODO: Al principio, el analizador sintáctico se encuentra en una configuración con $w\$$ en el búfer de entrada, y el símbolo inicial S de G en la parte superior de la pila, por encima de \$. El programa en la figura 4.20 utiliza la tabla de análisis sintáctico predictivo M para producir un análisis sintáctico predictivo para la entrada. □

establecer *ip* para que apunte al primer símbolo de w;
establecer X con el símbolo de la parte superior de la pila;
while ($X \neq \$$) { /* la pila no está vacía */
    if (X es a) sacar de la pila y avanzar *ip*;
    else if (X es un terminal) *error*();
    else if ($M[X, a]$ es una entrada de error) *error*();
    else if ($M[X, a] = X \rightarrow Y_1 Y_2 \cdots Y_k$) {
        enviar de salida la producción $X \rightarrow Y_1 Y_2 \cdots Y_k$;
        sacar de la pila;
        meter $Y_k, Y_{k-1}, \ldots, Y_1$ en la pila, con Y_1 en la parte superior;
    }
    establecer X con el símbolo de la parte superior de la pila;
}

ANALISIS SINTÁCTICO PREDICTIVO

Gramáticas LL(1)

$$
\begin{aligned}
E &\rightarrow T\ E' \\
E' &\rightarrow +\ T\ E' \mid \epsilon \\
T &\rightarrow F\ T' \\
T' &\rightarrow *\ F\ T' \mid \epsilon \\
F &\rightarrow (\ E\ ) \mid \mathbf{id}
\end{aligned}
$$

	id	(	)	*	+	$
E	E->TE´	E->TE´				
E´			E´-> ε		E´->+TE´	E´->ε
T	T->FT´	T->FT´				
T´			T´-> ε	T´->*FT´	T´-> ε	T´-> ε
F	F-> id	F-> (E)				

Pila	Entrada	Salida
$ E	id + id $	E->TE´
$ E´T	id + id $	T->FT´
$E´T´F	id + id $	F->id
$E´T´id	id + id $	
$E´T´	+ id $	
$E´	+ id $	T´-> ε
$E´	+ id $	E´->+TE´
$E´T+	+ id $	
$E´T	id $	
…	…	

MODO PÁNICO

En caso de error, omitir símbolos hasta que aparezca un carácter de sincronización.

CASO TÍPICO

1. Todos los símbolos en SIGUIENTE(A) pertenecen al conjunto de sincronización de A.

No Terminal	Símbolo de Entrada					
	id	+	*	(	)	$
E	$E \to TE'$			$E \to TE'$		
E'		$E' \to +TE'$			$E' \to \epsilon$	$E' \to \epsilon$
T	$T \to FT'$			$T \to FT'$		
T'		$T' \to \epsilon$	$T' \to *FT'$		$T' \to \epsilon$	$T' \to \epsilon$
F	$F \to$ id			$F \to (E)$		

PRIMERO(F) = PRIMERO(T) = PRIMERO(E) = { (, id }
PRIMERO(E´) = { +, ε }, PRIMERO(T´)= { *, ε }

SGTE(E) = SGTE(E') = {), $ }
SGTE(T) = SGTE(T') = { +,), $ }, SGTE(F) = { +, *,), $ }

ANALISIS SINTÁCTICO PREDICTIVO

Recuperación de Errores

$$
\begin{aligned}
E &\rightarrow T\ E' \\
E' &\rightarrow +\ T\ E' \mid \epsilon \\
T &\rightarrow F\ T' \\
T' &\rightarrow *\ F\ T' \mid \epsilon \\
F &\rightarrow (\ E\ ) \mid \mathbf{id}
\end{aligned}
$$

No Terminal	PRIMERO	SIGUIENTE
E	(, id	) , $
E'	+ , ε	) , $
T	(, id	+ ,) , $
T'	* , ε	+ ,) , $
F	(, id	* , + ,) , $

NO TERMINAL	SÍMBOLO DE ENTRADA					
	id	+	*	(	)	$
E	$E \rightarrow TE'$			$E \rightarrow TE'$	sinc	sinc
E'		$E \rightarrow +TE'$			$E \rightarrow \epsilon$	$E \rightarrow \epsilon$
T	$T \rightarrow FT'$	sinc		$T \rightarrow FT'$	sinc	sinc
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow \mathbf{id}$	sinc	sinc	$F \rightarrow (E)$	sinc	sinc

ANALISIS SINTÁCTICO PREDICTIVO

Recuperación de Errores

ESTRATEGIA

1. Si M[A,a] = no definido (esta vacía la entrada), omitir a.
2. Si M[A,a] = sinc, se elimina A de la pila.
3. Si top() es un terminal b, y b≠a, entonces eliminar b de la pila.

PILA	ENTRADA	COMENTARIO
$E\ \$$	$)\ \mathbf{id} * + \mathbf{id}\ \$$	error, omitir)
$E\ \$$	$\mathbf{id} * + \mathbf{id}\ \$$	**id** está en PRIMERO(E)
$TE'\ \$$	$\mathbf{id} * + \mathbf{id}\ \$$	
$FT'E'\ \$$	$\mathbf{id} * + \mathbf{id}\ \$$	
$\mathbf{id}\ T'E'\$$	$\mathbf{id} * + \mathbf{id}\ \$$	
$T'E'\ \$$	$* + \mathbf{id}\ \$$	
$* FT'E'\ \$$	$* + \mathbf{id}\ \$$	
$FT'E'\ \$$	$+ \mathbf{id}\ \$$	error, $M[F, +] = \text{sinc}$
$T'E'\ \$$	$+ \mathbf{id}\ \$$	Se sacó F
$E'\ \$$	$+ \mathbf{id}\ \$$	
$+ TE'\ \$$	$+ \mathbf{id}\ \$$	
$TE'\ \$$	$\mathbf{id}\ \$$	
$FT'E'\ \$$	$\mathbf{id}\ \$$	
$\mathbf{id}\ T'E'\ \$$	$\mathbf{id}\ \$$	
$T'E'\ \$$	$\$$	
$E'\ \$$	$\$$	
$\$$	$\$$	

Grammar #3:

Procesar una gramática que recibe frases palíndromo. **Los espacios no cuentan.**

Ejemplos:

pop

a man a plan a canal panama

Solution: para alfabeto={a,b,c}. Generalizar.

<palindrome> ::= a <palindrome> a | b <palindrome> b | c

<palindrome> c <palindrome> ::= <letter>

<letter> ::= a | b | c

Below is a sample BNF grammar:

S -> - FN | FN

FN -> DL | DL . DL

DL -> D | D DL

D -> 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

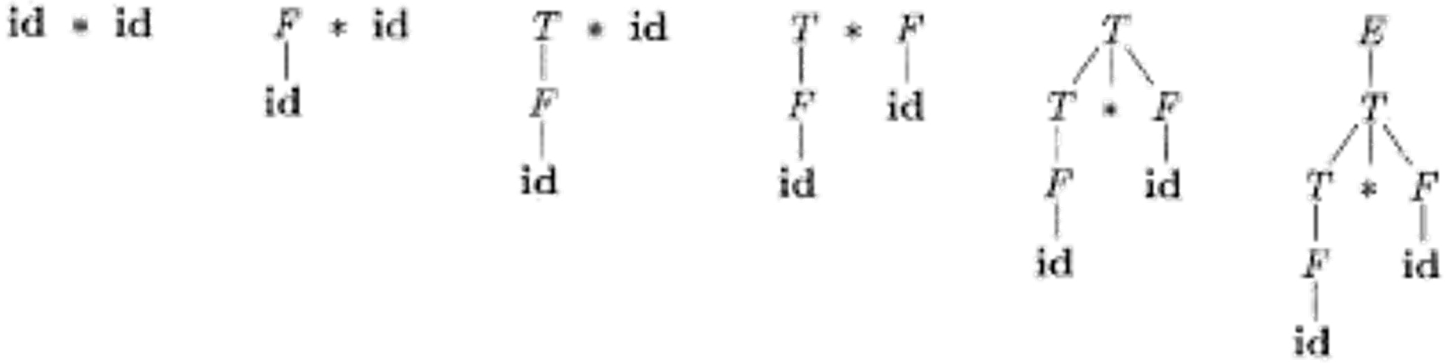
ANALISIS SINTÁCTICO ASCENDENTE

Reducciones

Construcción del árbol de análisis sintáctico en sentido ascendente: se comienza desde la cadena de entrada y se busca llegar al símbolo inicial de la gramática.

Reducción: Reducir la cadena de entrada al símbolo inicial.

Id + id => F * id => T * id => T * F => T => E
E=> T => T *F => T * id => F * id => id + id



ANALISIS SINTÁCTICO ASCENDENTE

Mangos

- Un mango de una cadena es una subcadena que concuerda con el lado derecho de una producción y cuya reducción al no terminal del lado izquierdo de la regla de producción representa un paso a lo largo de la inversa de una derivación por la derecha.
- Formalmente, un mango de una forma de frase derecha γ es una regla $A \rightarrow \beta$ y una posición de γ donde la cadena β podría encontrarse y sustituirse por A para producir la forma de frase derecha previa en una derivación por la derecha de γ.
- Es decir, si $S \Rightarrow^* \alpha Aw \Rightarrow^* \alpha\beta w$, entonces $A \rightarrow \beta$ si la posición que sigue de α es un mango de $\alpha\beta w$. La cadena w a la derecha del mango contiene sólo símbolos terminales.

Ejemplo	a b b c d e	mango posición 2 (regla A → b)
S → aABe	a A b c d e	mango posición 2 (regla A → Abc)
A → Abc	a A d e	mango posición 3 (regla B → d)
A → b	a A B e	mango posición 1 (regla S → aABe)
B → d	S	

ANALISIS SINTÁCTICO ASCENDENTE

Poda de Mangos

Para la gramática :

$E \rightarrow E + T \mid T$
$T \rightarrow T * F \mid F$
$F \rightarrow (E) \mid id$

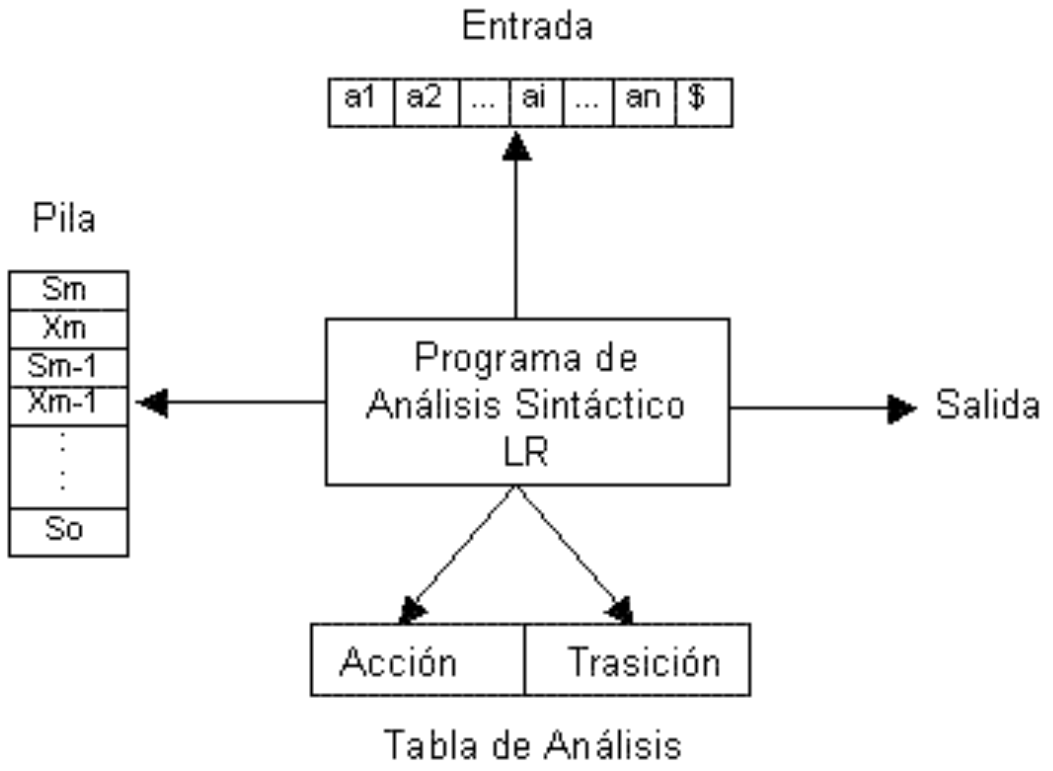
Una **reducción** es:

$id * id \Rightarrow F * id \Rightarrow T * id \Rightarrow T * F \Rightarrow T \Rightarrow E$

Forma de frase derecha	Mango	Reducción de la producción
$\mathbf{id}_1 * \mathbf{id}_2$	$\mathbf{id}_1$	$F \rightarrow \mathbf{id}$
$F * \mathbf{id}_2$	F	$T \rightarrow F$
$T * \mathbf{id}_2$	$\mathbf{id}_2$	$F \rightarrow \mathbf{id}$
$T * F$	$T * F$	$E \rightarrow T * F$

ANALISIS SINTÁCTICO ASCENDENTE

Analizador Sintáctico LR



Si w está en L(G), la reducción de un análisis sintáctico ascendente, en otro caso, Error.

ANALISIS SINTÁCTICO ASCENDENTE

Analizador Sintáctico LR

(1) $E \rightarrow E + T$
(2) $E \rightarrow T$
(3) $T \rightarrow T * F$
(4) $T \rightarrow F$
(5) $T \rightarrow (E)$
(6) $F \rightarrow \mathbf{id}$

Estado	Acción						ir_A		
	id	+	*	(	)	$	*E*	*T*	*F*
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

Agregar el estado 0 en la pila

hacer que a sea el primer símbolo de $w\$$;
while(1) { /* repetir indefinidamente */
    hacer que s sea el estado en la parte superior de la pila;
    if (ACCION[s, a] = desplazar t) {
        meter t en la pila;
        hacer que a sea el siguiente símbolo de entrada;
    } **else if** (ACCION[s, a] = reducir $A \rightarrow \beta$) {
        sacar $|\beta|$ símbolos de la pila;
        hacer que el estado t ahora esté en la parte superior de la pila;
        meter ir_A[t, A] en la pila;
        enviar de salida la producción $A \rightarrow \beta$;
    } **else if** (ACCION[s, a] = aceptar) break; /* terminó el análisis sintáctico */
    else llamar a la rutina de recuperación de errores;